SaveAlife BLR — Submission Document

Nov 8, 2026 · @Mrinal

IBM Bobathon: Build for Bengaluru · Theme 1, Smarter Streets Better Living

The problem

An ambulance stuck in Bengaluru traffic is a civic failure that kills. Road-accident victims have roughly an hour — the golden hour — to reach the right hospital, and in this city the ambulance often spends most of it crawling through the same jam that caused the delay in the first place.

Who faces it. The bystander who stops at a crash and does not know what to do. The dispatcher who has to pick a hospital and a route with no live picture of the roads. The ER that learns about the patient when the doors open.

What needs to improve. Three things, in order: the dispatcher needs to see which roads are actually moving right now, not which ones are usually fastest; the system needs to choose the hospital and the transport — road or air — on the patient's condition and the real road state, and say why; and the hospital needs to be ready before the patient arrives.

This is Theme 1, Smarter Streets Better Living: agentic AI applied to the one civic trip where minutes decide survival.

The prototype

SaveAlife BLR is a working, deployed web application with three screens and an agentic dispatch layer. A bystander reports a crash; within seconds a dispatcher sees a recommendation with the reasoning behind it, confirms it, and the hospital is pre-alerted before the vehicle moves.

Bystander screen (phone-sized). Report form prefilled with location; what happened, how many hurt, awake / breathing / bleeding; one tap to add a scene photo (five synthetic photos per scenario); after sending, a severity badge, an ETA line, and a first-aid checklist whose ticks appear on the ER timeline. A cartoon demo ID card stands in for identity.

Dispatch screen (the judge's view). A real Bengaluru map with the incident, six fixed traffic cameras showing LOW / HIGH / BLOCKED with looping clips, every candidate hospital, and all candidate routes drawn on real road geometry. Two reasoning panels side by side: the deterministic rule and the watsonx Orchestrate agent. A Confirm button — nothing moves without a human. After confirm, an eight-step sequence: incident pulses, hospital highlights, route highlights, vehicle leaves the hospital, loads the patient at the

scene, returns with the corridor junctions turning green, arrives, returns to base. Clicking the incident opens the photo album. A routes-considered panel gives one-line reasons for every rejection.

ER screen. Patient header, live vitals and ECG, the timeline from report to arrival, an intake form prefilled by the handoff step, Accept Handoff.

Five scenarios are selectable on any screen; all three screens follow the same active incident. The whole thing runs on IBM Cloud Code Engine at a public URL and offline on a laptop in simulation mode.

How the dispatch decision is made

The agent fuses two data sources — traffic cameras and the Routes API — and applies one rule that any dispatcher could read out loud.

Triage. Severity comes from the bystander's answers (unconscious or not breathing → P1; bleeding or a limb injury → P2; otherwise P3) and from the scene photo, read by a vision model on watsonx.ai for posture (lying still / sitting / standing), traffic density and vehicles. The more severe of the two wins. Severity sets the arrival window: P1 25 min, P2 45 min, P3 90 min. The need — neuro, trauma, ortho or general — comes from the mechanism of injury.

Cameras. Six fixed cameras at real junctions (Madiwala, Agara, Sarjapur Jn, Bommanahalli, Koramangala Sony World, HSR BDA). Each reports a vehicle count and a state: LOW under 80, HIGH 80–250 (adds delay in proportion to the count), BLOCKED (lane closed or stalled — any route through it is rejected outright).

Routes. Three candidate routes per scenario, each to a named hospital, with real road geometry and a base time from the Google Routes API. The ambulance comes *from* the hospital and returns *to* it, so the ETA is the round trip: out to the scene, back with the patient. For P1/P2 a green corridor on the return leg saves 1.5 min per junction.

Hospitals. Six real hospitals with specialties and a helipad flag. A hospital is usable only if it has the needed specialty and has an ambulance free in that scenario.

The rule.

1. Reject any route through a BLOCKED camera, to a hospital without the specialty, or to a hospital with no free ambulance.
2. Of the rest, take the fastest whose ETA (with corridor, for P1/P2) is inside the window.
3. If nothing passes and the patient is P1, propose the helicopter: HAL Airport → scene → nearest helipad hospital with the specialty.
4. If nothing passes and the patient is P2 or P3, take the fastest usable road and flag the delay to the hospital. A P3 never gets a helicopter.

5. A human dispatcher confirms before anything moves, and can override air to road.

The rule runs in Python and is the ground truth the tests assert on. The watsonx Orchestrate agent is given the same five tools and the same rule in its instructions, reasons independently, and its answer is shown beside the rule's with an "agrees" or "differs" chip. In all five scenarios it agrees.

The five scenarios

Each scenario changes one or two things and leaves the rest fixed, so the judge can see exactly what moved the decision. Five scenarios end at four different hospitals.

	Where · when	Cameras	Patient	Decision	Minutes saved vs route A
S1 One route blocked	Silk Board, under the flyover · 18:30	Madiwala **BLOCKED** (stalled truck), Koramangala HIGH, Agara HIGH, rest LOW	Car vs auto. Man sitting on the kerb holding his arm, awake. **P2, ortho**	**Road from Fortis via C**, corridor on return. 18.5 min.	13.6
S2 All roads fail, critical	Outer Ring Road near Agara Lake · 18:45	Agara **BLOCKED** (the crash itself), all other five HIGH	Bus vs truck. Man lying on the road, not moving, head injury. **P1, neuro**	**Air: HAL → scene → St John's helipad.** 14.7 min.	57.5
S3 Same crash, minor	Identical to S2	Identical to S2	Same bus and truck. Man standing, talking, holding one hand. **P3, general**	**Road from Apollo via C.** 56.8 min, no corridor.	5.2
S4 Clear roads, critical	Hosur Road near Bommanahalli · 06:10	All six LOW	Bike skid. Rider on his side, not responding, helmet off. **P1, neuro**	**Road from NIMHANS via A**, corridor anyway. 17.0 min.	0 (A is the default)
S5 Jam clears mid-trip	Sarjapur Road near Koramangala · 17:50	Sarjapur Jn HIGH 200, Koramangala HIGH 140, HSR HIGH, Madiwala HIGH; **Sarjapur and Koramangala drop to LOW during the return leg**	Bike behind a car. Rider against the divider, leg out, cannot move it. **P2, ortho**	**Road from Manipal via B**, corridor; passes the 45 min window by about a minute. ETA revised down live.	3.0

S1 — the agent finds the open road. Route A to St John's is the shortest but runs through Madiwala, where the camera shows a stalled truck across the lanes: rejected outright, not just slowed. Route B also goes to St John's, but St John's has no ambulance free this evening. Route C to Fortis passes. A system that only looked at distance would have sent the ambulance into the blockage.

S2 — the helicopter, and only because nothing else works. Agara is blocked by the crash itself; the other two routes are 60 and 67 minutes round trip against a 25-minute P1 window, even with a corridor. Every road option fails, the patient is critical, so the agent proposes air: HAL Airport to the scene in six minutes, then to St John's helipad. NIMHANS has neuro but no helipad, and the trace says so. The dispatcher confirms; the dispatcher can also override to road.

S3 — same crash, opposite answer. This is the pair that proves the agent is reasoning about the patient, not the traffic. Cameras, vehicles, blocked junction, route timings for A and B: identical to S2. The man is standing and talking, so he is P3 with a 90-minute window. B to St John's and C to Apollo both pass; C is faster. No corridor, no helicopter. A P3 does not get a helicopter however bad the jam is.

S4 — clear roads, but still a corridor. Every camera is LOW at 6 am. The rule still rejects Fortis (no neuro) and St John's (no ambulance free), and sends the NIMHANS ambulance — the neuro hospital — with a green corridor on the return, because a P1 patient gets every minute the city can give even when the roads are empty. Route A is both the default and the choice, so minutes saved reads zero; the gain is the corridor and the right hospital.

S5 — the decision is live, not one-shot. Both St John's routes are out (no ambulance free), leaving Manipal via Sarjapur Jn at 43.8 minutes against a 45-minute P2 window — it passes only because of the corridor. The ambulance goes out. On the return leg the Sarjapur and Koramangala cameras drop from HIGH to LOW; the agent re-checks and the trace shows the ETA falling. The timeline logs "Dispatch agent re-evaluated". Nothing about the route changes, but the hospital's expected arrival does, and the ER sees it.

To run a scenario: open the Dispatch screen, click S1–S5, click **Send bystander report** (or send it from the Bystander screen with a photo), read both reasoning panels, click **Confirm**, and watch the sixty-second sequence. Switch to the ER screen at any point.

Architecture

Everything runs on IBM Cloud; the only outside service is Google's Routes API for road geometry and live times.

Layer	What it does	Technology
Dispatch agent	Reads incident, cameras, routes, hospitals and air option through five tools; reasons to a recommendation with reasons; shown beside the rule	**watsonx Orchestrate**, native agent, five OpenAPI tools, gpt-oss-120b (Orchestrate's recommended agent model)
Triage and vision	Classifies the bystander's text; reads the scene photo for posture, traffic and vehicles	**watsonx.ai**: Granite 4 H Small for text, Llama 4 Maverick for images
Backend	Scenario data, the decision rule, two-leg ETAs, corridor plan, vehicle clock, synthetic vitals, the tool endpoints the agent calls	FastAPI, Python 3.11
Screens	Bystander, Dispatch, ER — hand-built HTML/JS, Leaflet map on OpenStreetMap tiles with a light/dark toggle, live ECG on canvas	No framework
Hosting	Container built from source, public HTTPS URL, scales to one instance	**IBM Cloud Code Engine**
Roads and times	Real polylines and live durations for 15 routes, fetched once and cached so the demo works offline	Google Routes API
Cameras	Six fixed junctions, three states, looping synthetic clips	Simulated in this build; the interface expects a count and a state per camera
Build	Every deployment, tool, agent and wiring task written by Bob from a spec, sessions exported	**IBM Bob 2.0**

Request flow. Bystander report → `/api/incident/report` → triage (rules + photo read) → decision rule → in parallel, a background call to the Orchestrate agent, which calls back into `/api/tools/{scenario}/…` on the same deployment → both results on the dispatch screen → dispatcher confirms → sixty-second sequence driven by the server clock, polled once a second by all three screens.

Why the agent is on gpt-oss-120b and not Granite. Orchestrate's own model list marks Granite as not recommended for agents, and in testing Granite 4 H Small returned empty messages to the tool-calling prompt. gpt-oss-120b is Orchestrate's default and what IBM's

own stock agents in the instance use. Granite still does the triage and photo work on watsonx.ai. This is stated plainly rather than hidden.

The Bob story

Bob built the entire IBM-platform layer of this project from written specs, in four tasks, with the session exported for each in `bob_sessions/`. The UI and the simulation engine were hand-built first so Bob had a stable, tested codebase to extend — every Bob task ran the full test suite and reported green before finishing.

Task	Spec given to Bob	What Bob produced	Tests after
1 · Deploy	Prepare for Code Engine without touching app logic	Dockerfile, .dockerignore, deploy script, README deploy section, PORT handling	19 pass
2 · Tools	Stateless tool endpoints keyed by scenario, plus the OpenAPI file Orchestrate imports	`app/routers/tools.py` with five endpoints and LLM-facing descriptions, OpenAPI exporter, import script, 29 new tests	48 pass
3 · Agent	Native agent YAML with the decision rule as instructions, import script, five-scenario test harness against the Orchestrate REST API	`orchestrate/dispatch_agent.yaml`, `wxo_import_agent.ps1`, `wxo_test_agent.py`; then a follow-up to correct the chat endpoint (runs → poll → thread messages)	48 pass; agent 5/5 scenarios match the rule
4 · Wiring	Call the agent in a background thread on every report, keep the rule as ground truth, show both on screen	`app/services/wxo_agent.py`, settings flag, dispatch panel with agrees/differs chip, 2 new tests. Bob spotted that its own local import would defeat the test's monkeypatch and fixed it unprompted	50 pass

Every task was a single pasted spec; debugging and design happened outside Bob so that each session is a clean record of what Bob did. Total Bob usage: 4 Bobcoins of 50.

Two things from the sessions worth a judge's attention: Bob wrote the five tool descriptions that the agent uses to decide which tool to call, and they worked first time; and in task 4 Bob caught a test-isolation bug in its own code before reporting.

Impact

The one measurable outcome: minutes saved from report to hospital arrival, against dispatching on the default (shortest) route. It is computed per incident and shown on the dispatch screen as the sequence runs. These are scenario-modelled comparisons on synthetic traffic, not measured real-world savings: in S1–S3 the default route is blocked, so the baseline is the time a distance-only dispatcher would have committed to before discovering the blockage.

Scenario	Default route A	Chosen	Minutes saved
S1	32.1 (blocked — would not have arrived)	18.5	13.6
S2	72.2 (blocked)	14.7 by air	57.5
S3	62.0 (blocked)	56.8	5.2
S4	17.0	17.0 (same route, with corridor and the right hospital)	0
S5	44.7	41.7, revised down live	3.0

Read it this way: in three of five scenarios the default dispatch would not have arrived at all, and in S2 no road option could get a P1 patient to a neuro-capable hospital inside the window. The number that matters most is not the average but the S2 case, where the difference is whether the patient arrives at all.

What it costs to run. Measured on this build's IBM Cloud account: one dispatch decision is about 1,600 tokens on the Orchestrate agent and under 1,000 on Granite and the vision model — roughly a fifth of a US cent. The deployed app used 0.2 GB-hours on Code Engine for a day of testing. A city running a thousand incidents a day would spend a few dollars. Routes API calls are cached per route, so the map and the live times cost nothing after the first fetch.

Scalability. Cameras, hospitals, junctions and routes are data, not code: adding a corridor is adding rows. The agent's tools are stateless and keyed by scenario, so the same agent serves any number of concurrent incidents. The expensive step — the vision read — happens once per photo, not per poll.

Privacy and human review

The hackathon rules ask safety-focused projects to design in privacy and human review. Both are structural here, not bolted on.

Human review. The agent recommends; it never dispatches. Air dispatch and green-corridor requests are gated behind a dispatcher's Confirm button, and the dispatcher can override air to road. Both the rule's reasoning and the agent's are on screen before the click, with every rejected option and its reason. If the agent and the rule ever disagree, the screen says so and the rule applies.

Privacy. No real person appears anywhere. The patient is a cartoon demo ID (SAL-DEMO-0001); vitals, ECG and timeline are synthetic; the twenty-five scene photos are illustrations with no faces, plates or wounds; the camera clips are synthetic. The scene photo is analysed on the server and only the structured read (posture, traffic, vehicles) is forwarded — the hospital never sees the image. No government ID, health record, or children's data is used, in line with rule 8.3. Everything synthetic is labelled on screen as such, in line with rule 8.4.

What a real deployment would add. Consent and retention rules for scene photos; the hospital's own patient-matching once the ambulance arrives; audit logging of every confirm and override.

Links, how to run, tests, limits

Live prototype: https://savealife-blr.2gwnl9m982ir.us-south.codeengine.appdomain.cloud **Source:** https://github.com/MrinalDesai/savealife-blr **Video:** https://youtu.be/kk92x39pzfc **Bob sessions:** `bob_sessions/` in the repo, one export per task.

Run locally

```
git clone https://github.com/MrinalDesai/savealife-blr
cd savealife-blr
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env        # SIMULATION=true needs no keys
uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000/ and pick a screen. With IBM and Google keys in `.env`, `python scripts/preflight.py` verifies every service, `python scripts/fetch_routes.py` pulls real road geometry, and `AGENT_LIVE=true` turns on the Orchestrate panel.

Tests: 51, run with `pytest -q`. They cover the full flow for every scenario and its expected decision, the S2/S3 contrast, override, the S5 re-evaluation, the eight animation phases, blocked-route rejection, all five tool endpoints, and the agent wiring with the agent mocked out.

What is prototype and what is operational

This is a working prototype of the decision and coordination layer, not emergency infrastructure. The cameras, patient data, vitals, scene photos and camera clips are synthetic and labelled so on screen; the road geometry and live times are real; the agent, the tools, the deployment and the human-confirm gate are the real mechanism a city would run. Known limits:

- Traffic cameras are simulated; the camera interface expects a count and a state per junction, which is what a real feed would supply.
- Route base times are a scenario snapshot; the live Routes API time is fetched and shown beside it.
- The Orchestrate agent runs on gpt-oss-120b, not Granite (see Architecture).
- One active incident at a time in the demo server; the tools and the agent are stateless.
- Helicopter ambulances are rare in Bengaluru today; the air option shows what the system would do when they exist, and which corners would justify them.

Built by Mrinal Desai, solo, 7–8 November 2026.